The 90-Day Cascade Roadmap

Building one production-grade AI serving platform, one commit a day

Constraints respected: 45 min/day, 90 days, runs alongside your Deep Learning Specialization, one evolving repo, daily commits, ~85-90 total commits by the end.

The project: you're building **Cascade** — a recommendation-serving platform structurally identical to what backs a Reels-style feed or Netflix's row ranking, sized to run on your laptop. It maps directly onto the video-feed scoring pipeline you already own at your internship, so skills transfer in both directions.

How to read this document: each week opens with the *why* — the design rationale, the exact resources (with chapters/sections named), the mistakes beginners make, and how it connects to Google-scale systems. That content is scoped once per week because it doesn't change day to day. Under each week header is a 5–7 row daily table — every single row is a real coding task, ending in a real commit. Skip nothing in the weekly "why," but treat the daily table as your actual to-do list.

WEEK 1 — Foundations: Repo, FastAPI, Layered Architecture

Why this matters: Every outage post-mortem eventually traces back to a violated architectural boundary. You need to feel *why* layering exists — routing, business logic, and data access as separate concerns — before you can be trusted to design a service, and that habit is cheapest to build on day one, on a blank repo.

Concepts to master (essential): separation of concerns, the request/response lifecycle, statelessness, Pydantic validation, structured error handling.

Resources:

- FastAPI official tutorial, "First Steps" through "Handling Errors" (https://fastapi.tiangolo.com/tutorial/) — stop there this week; you'll return for Security and async sections later.
- Skip: any "system design for beginners" video series — architecture sticks once you have real code to hang it on, not before.

Common mistakes: putting business logic directly inside route handlers "because it's just one endpoint" — this habit is what makes a codebase unmaintainable at 50 endpoints; kill it in week 1.

Connects to Google-scale systems: this routing/logic/data separation is the exact shape of every internal RPC service (Stubby/gRPC-based) — FastAPI is the on-ramp to that mental model.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
1	Repo hygiene	Init git repo, add `.gitignore`, `pyproject.toml`/`requirements.txt`, README skeleton, virtualenv	—	`chore: initialize repository structure`	4	Clean repo scaffold, first commit exists
2	Minimal API	Install FastAPI + uvicorn, create `main.py` with a `/health` endpoint	—	`feat: add FastAPI app with health check`	2	`uvicorn main:app` runs, `/health` returns 200
3	Layering	Split `main.py` into `routers/`, `services/`, `schemas/` folders; move health check into a router	Refactor: no logic in `main.py` beyond app init	`refactor: separate routing from business logic`	4	Same behavior, cleaner structure
4	Request/response schemas	Add Pydantic models for `ScoreRequest`/`ScoreResponse`; add stub `POST /score` returning a shuffled candidate list	—	`feat: add /score endpoint with Pydantic schemas`	3	`/docs` shows Swagger UI with typed schemas
5	Validation & errors	Add input validation (empty candidate list, invalid types) with custom `HTTPException` handlers	Test: manual curl of bad payloads	`feat: add input validation and error handling`	3	Bad requests return clean 4xx, not 500s
6	Testing basics	Add `pytest` + `TestClient`; write 3 tests covering `/health` and `/score` happy/error paths	Test: run `pytest -v`	`test: add unit tests for core endpoints`	2	All tests green
7	Logging + review	Add structured logging (Python `logging` module, JSON-ish format) to every route; clean up any dead code	Refactor: remove any leftover print statements	`feat: add structured logging; weekly cleanup`	3	Every request logs method, path, status, latency

Weekly review: Your repo now has a real three-layer FastAPI app with tests and logging — no model, no DB yet, that's intentional. A Google reviewer would flag: are your routers thin? Is any config hardcoded that should be an env var? Stretch goal (optional): add a `pyproject.toml` with `ruff`/`black` configured.

WEEK 2 — PostgreSQL + SQLAlchemy

Why this matters: Your two-tower model needs user embeddings, item metadata, and interaction history from somewhere real, not a Python dict. Relational databases remain the source-of-truth layer for recommendation metadata even when the serving path uses caches on top.

Concepts to master (essential): schema design, primary/foreign keys, indexes and why they matter for latency, the N+1 query problem, SQLAlchemy 2.0's `select()`-style Core usage.

Resources:

- SQLAlchemy 2.0 tutorial, "Working with Data" section only (https://docs.sqlalchemy.org/en/20/tutorial/data.html) — skip the ORM-relationship deep chapters until Week 3.
- "Use The Index, Luke," just "Anatomy of an Index" (https://use-the-index-luke.com/sql/anatomy) — the single best page on real-world query latency you'll find.

Common mistakes: fetching entire tables into Python and filtering in application code instead of pushing the filter into SQL — the most common junior-engineer performance bug.

Connects to Google-scale systems: feature stores (Google's internal ones, or open equivalents like Feast) industrialize exactly this pattern — structured storage of user/item features queried at low latency.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
8	DB connection	Run Postgres in a local container; add `psycopg2`/`asyncpg` config via env vars	—	`feat: add Postgres connection config`	2	App connects to a live Postgres instance
9	Schema design	Define SQLAlchemy models: `User`, `Item`, `Interaction` (user_id, item_id, event_type, timestamp)	—	`feat: add SQLAlchemy models for core entities`	1	Tables created via metadata
10	Dependency injection	Add a DB session as a FastAPI dependency (`Depends(get_db)`)	Refactor: routers now receive `db: Session`	`feat: add DB session dependency`	3	Routes can query the DB cleanly
11	CRUD basics	Implement `create_user`, `log_interaction` helper functions in a `repository.py`	Test: unit test each CRUD function	`feat: add CRUD operations for interactions`	2	Functions insert/read real rows
12	Real queries	Replace the shuffled candidate list in `/score` with a real query: user's last 20 interactions	—	`feat: query real interaction history in /score`	2	`/score` reflects actual DB state
13	Indexing	Add an index on `(user_id, timestamp)`; run `EXPLAIN` before/after and note the plan change	—	`perf: add index on interactions table`	1	Query plan shows index usage
14	DB testing + review	Add integration tests using a transaction-rollback fixture so tests don't pollute real data	Refactor: extract test DB fixture	`test: add database integration tests`	3	Tests pass without leaving residue in DB

Weekly review: Repo now has real persistence and real queries backing `/score`. A reviewer would ask: is the session properly closed after each request? Did you commit a migration path, or are tables created ad hoc? Technical debt to flag: no Alembic yet — that's fine, note it and move on.

WEEK 3 — Authentication

Why this matters: A real serving system needs to know *who* is asking, both for personalization and abuse prevention — and this is where you first feel the tension between "secure" and "fast" that defines production auth.

Concepts to master (essential): stateless auth via JWT, password hashing (bcrypt), authentication vs. authorization.

Resources:

- FastAPI tutorial, "Security" section in full (https://fastapi.tiangolo.com/tutorial/security/) — the best JWT-with-FastAPI walkthrough available; no need to look elsewhere.

Common mistakes: storing plaintext passwords "just for now, I'll fix it later" — never; build the habit correctly from line one.

Connects to Google-scale systems: internal services use mTLS/service identity instead of JWTs, but the shape — every request carries verifiable identity, nothing trusted by default — is the same zero-trust reasoning at Google's scale.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expect Outcor
15	Password hashing	Add `passlib`/bcrypt hashing utility; add `hashed_password` column to `User`	—	`feat: add password hashing utility`	2	Passwo stored plainte
16	Registration	Add `POST /register` endpoint	Test: register + verify hash stored	`feat: add user registration endpoint`	2	New us persist hashed passwo
17	JWT utility	Add `create_access_token`/`decode_access_token` helpers using `python-jose`	—	`feat: add JWT utility functions`	1	Tokens encode correct
18	Login	Add `POST /login`, verify password, return a JWT	Test: valid/invalid credentials	`feat: add login endpoint with JWT issuance`	2	Valid lo returns token
19	Protecting routes	Add `OAuth2PasswordBearer` dependency; protect `/score`	—	`feat: protect /score endpoint with JWT auth`	2	Unauth calls ge
20	Identity from token	Derive `user_id` from the decoded token instead of trusting the request body	Refactor: remove `user_id` from `ScoreRequest`	`security: derive user identity from token`	2	Client longer anothe ID
21	Auth tests + review	Add tests for valid, invalid, expired, and missing tokens	Test suite for full auth flow	`test: add authentication test suite`	2	All auth cases c

Weekly review: Repo now has real identity end to end. Reviewer questions: is the JWT secret in an env var, not hardcoded? Is token expiry actually enforced, or just decoded? Stretch goal (optional): add refresh tokens.

WEEK 4 — Docker + Docker Compose

Why this matters: "Works on my machine" is the most expensive sentence in engineering. Containerization turns a multi-service system into a single reproducible command for anyone, including future-you.

Concepts to master (essential): images vs. containers, Dockerfile layering/caching, Compose networking, volumes for persistence.

Resources:

- Docker's "Get Started," Parts 1–3 only (https://docs.docker.com/get-started/) — skip Kubernetes-adjacent later parts.
- Docker Compose "Quickstart" (https://docs.docker.com/compose/gettingstarted/).

Common mistakes: hardcoding `localhost` as the Postgres host — inside Compose's network the hostname is the *service name*, not `localhost`. This trips up nearly everyone the first time.

Connects to Google-scale systems: Docker is the direct ancestor of Google's internal container model (Borg), later open-sourced as Kubernetes — every production AI system you'll ship runs in some descendant of this.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
22	Dockerfile	Write a Dockerfile for the app	—	`feat: add Dockerfile for app`	1	`docker build` succeeds
23	Image slimming	Add `.dockerignore`; try a multi-stage build to cut image size	—	`perf: optimize Docker image size`	2	Smaller final image
24	Compose basics	Write `docker-compose.yml` with `app` + `postgres` services	—	`feat: add docker-compose for app and postgres`	1	`docker compose up` brings up both
25	Health checks	Add `healthcheck` + `depends_on: condition: service_healthy`	—	`feat: add service health checks in compose`	1	App waits for DB readiness before starting
26	Persistence + env	Add named volumes for Postgres data; move secrets/config into `.env`	—	`feat: add persistent volumes and env config`	2	Data survives container restarts
27	Networking fixes	Fix any hardcoded `localhost` references; do a full teardown/rebuild test	Test: `docker compose down -v && up` clean run	`fix: correct service hostname resolution in compose network`	1	Fresh clone + one command works end to end
28	Docs + review	Write clear run instructions in README	—	`docs: add setup and run instructions`	1	Anyone can clone and run with zero extra steps

Weekly review: The whole system is now reproducible with one command. Reviewer question: does your Dockerfile rebuild dependencies on every code change (bad layer ordering), or is caching working? Technical debt to flag: no multi-service orchestration for a scaled/replicated setup yet — that's Week 9.

WEEK 5 — Concurrency: Async, Multithreading, Multiprocessing

Why this matters: This is the most misunderstood topic among strong ML engineers moving into systems work — and it's essential, not optional, for a scoring service that fetches features, calls a model, and logs an event, ideally not one after another.

Concepts to master (essential): the GIL and what it restricts, why `asyncio` helps I/O-bound work but not CPU-bound work, when multiprocessing is the right tool instead, `async`/`await` in FastAPI.

Resources:

- Real Python, "Async IO in Python: A Complete Walkthrough" — read only "The asyncio Package and async/await" and "The Asyncio Event Loop" sections (https://realpython.com/async-io-python/).
- Skip deep multiprocessing dives this week — the project task below is enough exposure for now.

Common mistakes: assuming `async` speeds up CPU-bound model inference — it doesn't. This exact confusion is the most common interview red flag for ML engineers claiming systems skills.

Connects to Google-scale systems: inference servers (TorchServe, Triton, Google's internal serving stack) separate async request handling from a dedicated worker pool for GPU/CPU-bound compute for exactly this reason.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
29	Async basics	Convert `/score` to `async def`; measure baseline latency	—	`refactor: convert score endpoint to async`	1	Endpoint still works, now async
30	Async DB	Migrate to an async SQLAlchemy engine (`asyncpg`)	Refactor: update repository functions to `async`	`feat: migrate to async SQLAlchemy engine`	3	DB calls no longer block the event loop
31	Concurrent I/O	Use `asyncio.gather` to run feature fetch + (stub) model scoring concurrently	—	`perf: parallelize feature fetch and scoring with asyncio.gather`	1	Two I/O calls run in parallel, not sequentially
32	Benchmarking	Add artificial latency (`asyncio.sleep`) to simulate a model call; benchmark concurrent vs. sequential	Test: simple timing script	`test: benchmark concurrent vs sequential fetch`	1	Concurrent path is measurably faster
33	Multiprocessing	Write a standalone script batch-scoring 10,000 fake users using `multiprocessing.Pool`	—	`feat: add multiprocessing batch scoring script`	1	Script runs, produces scored output
34	CPU-bound benchmarking	Benchmark the multiprocessing script vs. a single-process loop; document results	—	`docs: add concurrency benchmark results`	1	Clear numbers showing when parallelism wins
35	Async testing + review	Add `pytest-asyncio` tests for the async endpoints	Test suite pass	`test: add async endpoint test coverage`	1	Async paths are test-covered

Weekly review: You should now be able to explain, unprompted, why async wouldn't help a CPU-bound model call. Reviewer question: are you awaiting things you don't need to await (fake concurrency)? Stretch goal (optional): profile the event loop with `asyncio` debug mode.

WEEK 6 — Redis + Caching

Why this matters: Your "last 20 interactions" query will not survive real traffic at low latency. Caching is the difference between a 200ms and a 5ms response — and invalidation is famously one of the two hard problems in computer science.

Concepts to master (essential): cache-aside pattern, TTLs, invalidation strategies, when caching helps vs. hides a real problem.

Resources:

- Redis docs, "Data types" — Strings, Hashes, Sorted Sets only (https://redis.io/docs/latest/develop/data-types/).
- Redis-py quickstart (https://redis.io/docs/latest/develop/connect/clients/python/).

Common mistakes: caching without a TTL or invalidation plan "to keep it simple" — this is exactly how production systems end up silently serving days-stale data.

Connects to Google-scale systems: every large-scale recommender (YouTube, Reels, Netflix) caches in front of the model for this reason — inference is expensive, and most feeds don't need recomputing on every request.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
36	Redis setup	Add Redis to compose; connect via `redis-py`	—	`feat: add Redis service and client`	2	App can read/write Redis
37	Cache-aside	Implement cache-aside for `/score`: check cache before compute, write through after	—	`feat: add cache-aside pattern to /score`	1	Repeated calls hit cache, not DB
38	TTLs	Add a 60-second TTL; test expiry behavior	Test: expiry test with short TTL	`feat: add TTL-based cache expiration`	1	Stale entries expire correctly
39	Invalidation	Invalidate a user's cache entry when a new interaction is logged	—	`feat: add cache invalidation on interaction write`	1	Fresh interactions reflect immediately
40	Sorted sets	Add a Redis sorted set tracking item popularity, updated on each interaction	—	`feat: add popularity ranking via Redis sorted sets`	1	Popularity scores update live
41	Cold start	Use the popularity sorted set as a fallback ranker for users with zero history	Test: cold-start user path	`feat: add cold-start fallback ranking`	1	New users still get a reasonable ranked list
42	Observability + review	Add cache hit/miss logging	—	`feat: add cache hit/miss logging`	1	You can see cache effectiveness in logs

Weekly review: Reviewer question: what happens to correctness if a user's cache hasn't expired but their interactions changed? You should be able to answer this precisely now. Technical debt to flag: no metrics yet on hit rate — that lands in Week 11.

WEEK 7 — Message Queues + Celery

Why this matters: Right now, logging an interaction blocks the request until the DB write finishes. In production, logging and retraining triggers must never block the user-facing path — the core lesson of async task processing.

Concepts to master (essential): producer/consumer pattern, why queues decouple services in time, retries and idempotency, `Celery beat` for scheduled jobs.

Resources:

- Celery's "First Steps with Celery," done verbatim before touching your own project (https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html).

Common mistakes: writing background tasks that aren't idempotent, so a retry after a transient failure double-processes an event (e.g., double-counts a click). Always assume a task might run twice.

Connects to Google-scale systems: this producer/consumer decoupling is the same reasoning, at vastly larger scale, behind internal event-pipeline systems.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
43	Celery setup	Add Celery with Redis as broker	—	`feat: add Celery worker configuration`	2	Worker starts and connects to broker
44	Offload writes	Move interaction logging off the request path into a Celery task	Refactor: `/interact` enqueues instead of writing directly	`refactor: move interaction logging to background task`	2	Request returns instantly; write happens async
45	Idempotency	Add an idempotency key + retry policy to the logging task	Test: simulate duplicate task execution	`feat: add idempotent retry logic to logging task`	1	Duplicate task runs don't double-count
46	Scheduled jobs	Add `Celery beat` task recomputing popularity every 5 minutes	—	`feat: add scheduled popularity recomputation task`	1	Popularity refreshes on a schedule, not per-write
47	Failure handling	Add dead-letter logging for tasks that exhaust retries	—	`feat: add failure handling for background tasks`	1	Failed tasks are visible, not silently dropped
48	Resilience test	Kill the worker mid-traffic, verify queue backlog, restart and watch it drain	Test: manual resilience test, documented	`test: verify queue resilience during worker downtime`	1	System survives worker downtime without data loss
49	Docs + review	Document the task architecture (diagram or written description)	—	`docs: document background task architecture`	1	Clear record of how async tasks flow

Weekly review: You've now watched "the queue absorb an outage" firsthand — that's the entire point of this pattern. Reviewer question: is every task idempotent, or did you only fix the one you tested?

WEEK 8 — Kafka + Event-Driven Architecture

Why this matters: Celery/Redis is fine for task queues, but recommendation systems live on *event streams* — every impression and click is a training signal, and Kafka is the industry-standard backbone for capturing that stream durably at volume.

Concepts to master (essential): topics, partitions, producers/consumers, consumer groups, at-least-once delivery, why event logs beat direct DB writes for high-volume telemetry.

Resources:

- Confluent's "Kafka 101," first four videos only: Introduction, Topics, Partitions, Producers/Consumers (https://developer.confluent.io/courses/apache-kafka/events/) — genuinely better than raw Apache docs at this stage.
- Skip Kafka Streams/Connect entirely — out of scope here.

Common mistakes: treating Kafka like a queue (single consumer draining messages) instead of a log (multiple independent consumers reading the same stream at their own pace) — this misunderstanding causes badly misused partitioning.

Connects to Google-scale systems: this is structurally identical to how YouTube/Reels-scale recommenders capture watch-time and click signals feeding real-time features, offline training, and analytics simultaneously.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
50	Kafka setup	Add Kafka (KRaft mode, no Zookeeper needed) to compose	—	`feat: add Kafka service to docker-compose`	1	Kafka broker runs locally
51	Producer	Add a Kafka producer client; produce interaction events	—	`feat: add Kafka producer for interaction events`	1	Events appear on the topic (verify with CLI)
52	Replace Celery write	Replace the Celery interaction-write task with a Kafka `produce` call	Refactor: remove old Celery logging task	`refactor: publish interactions to Kafka topic`	2	Interactions now flow through Kafka
53	Consumer #1	Write a standalone consumer script that writes events to Postgres	—	`feat: add Kafka consumer writing to Postgres`	1	Consumer persists events reliably
54	Consumer #2	Write a second consumer updating the Redis popularity sorted set from the same topic	—	`feat: add second Kafka consumer for Redis updates`	1	Two independent consumers read the same log
55	Consumer groups	Configure consumer groups; test parallel consumption with 2 instances of one consumer	Test: verify no duplicate processing within a group	`feat: configure Kafka consumer groups`	1	Partitioned parallel consumption works
56	Testing + review	Add basic producer/consumer tests	—	`test: add Kafka producer/consumer test coverage`	1	Core event flow is test-covered

Weekly review: Reviewer question: if you added a third consumer tomorrow (e.g., a training-data exporter), would it require any change to the producer? (It shouldn't — that's the whole point of the log.) Budget note: expect Days 50–53 to spill a bit — Kafka's local setup is famously fiddly the first time.

WEEK 9 — Distributed Architecture: Replicas, CAP, Sharding, Load Balancing

Why this matters: Every service so far has assumed one instance of everything. Real systems run many replicas across machines that fail independently — and per your daily-coding constraint, we turn CAP/replication/sharding theory into small runnable experiments and design docs, not just reading.

Concepts to master (essential): CAP theorem (about behavior *during a partition*, not a permanent 3-way tradeoff), replication (leader/follower), sharding/partitioning strategies, load balancing basics.

Resources:

- "Designing Data-Intensive Applications" (Kleppmann) — Chapter 5 (Replication) and Chapter 6 (Partitioning) only. Skip Chapters 1–4 and 7–12 for this roadmap — this is the best resource on the topic, but reading it cover-to-cover would blow the time budget.
- Skip "CAP theorem explained" videos — they oversimplify into the "pick 2 of 3" myth that Kleppmann's chapters correctly complicate.

Common mistakes: memorizing "CP vs. AP" as a fixed database label. Real systems make this choice per-operation, not once for the whole system.

Connects to Google-scale systems: this is the daily vocabulary of a Google systems-design interview and the daily reality of Spanner/Bigtable-style infrastructure — this week has the highest interview-ROI in the roadmap.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
57	Load balancing	Add an Nginx reverse proxy in front of the API (single instance for now)	—	`feat: add Nginx reverse proxy`	1	Traffic routes through Nginx to the app
58	Scaling replicas	Scale the API to 3 replicas via `docker compose up --scale`; confirm Nginx load-balances across them	—	`feat: scale API to multiple replicas behind Nginx`	1	Requests are served by different replicas
59	Statelessness	Write a test that hits the app repeatedly and confirms no replica-specific in-memory state leaks into behavior	Test: statelessness verification	`test: verify service statelessness across replicas`	1	Behavior is identical regardless of which replica answers
60	CAP design doc	Write an ADR (architecture decision record) on CAP tradeoffs for Cascade specifically	—	`docs: add ADR on CAP theorem tradeoffs`	1	A one-page doc reasoning about your actual system, not the concept in the abstract
61	Replication design	Write an ADR on a Postgres leader/follower replication strategy; add a commented-out read-replica config stub	—	`docs: add replication strategy design`	1	Clear reasoning about read/write split tradeoffs
62	Sharding design	Write an ADR on sharding the `interactions` table by `user_id` hash; add a partition-key comment in the schema	—	`docs: add sharding strategy design`	1	You can explain your shard key choice and its failure modes

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
63	Partition simulation + review	Kill the Postgres container mid-request; observe the failure, then fix `/score` to fall back to cache-only mode gracefully	Test: partition simulation	`feat: add graceful degradation when Postgres unavailable`	2	System degrades instead of erroring outright

Weekly review: This is the highest-density week for interview prep. Reviewer question: in your ADRs, did you pick a concrete tradeoff and defend it, or just list both sides? A real design doc commits to a decision.

WEEK 10 — Data Pipelines + Scheduling

Why this matters: Model features (popularity, session-length averages) need periodic recomputation from raw event data — this is the "batch" half of a real ML system, complementing the online-serving half you've already built.

Concepts to master (essential): batch vs. streaming, DAG-based scheduling (why cron alone breaks down once tasks have dependencies), idempotent pipeline design.

Resources:

- Airflow's "Quick Start" + "Fundamental Concepts" only (https://airflow.apache.org/docs/apache-airflow/stable/tutorial/fundamentals.html) — stop there, skip the provider/plugin ecosystem docs.

Common mistakes: writing a pipeline task that isn't safely re-runnable, so a manual retry after a failure corrupts downstream data (e.g., double-counted averages).

Connects to Google-scale systems: this is the same reasoning behind internal orchestration systems that feed offline feature computation into online feature stores — keeping model inputs fresh without touching the serving path.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
64	Airflow setup	Add Airflow to your stack (standalone or via compose); confirm the UI loads	—	`feat: add Airflow service`	1	Airflow UI is reachable
65	DAG skeleton	Write an empty DAG with two placeholder tasks	—	`feat: add initial Airflow DAG skeleton`	1	DAG appears in the UI, runs as a no-op
66	Extract task	Implement task 1: pull the day's interactions from Postgres	—	`feat: add extract task to DAG`	1	Task pulls real rows
67	Transform task	Implement task 2: compute a feature (e.g., average session length this week)	—	`feat: add feature computation task to DAG`	1	Feature values are computed correctly
68	Dependencies	Wire `task1 >> task2`; run the full DAG end to end	Test: manual DAG trigger and verify	`feat: wire DAG task dependencies`	1	DAG completes both tasks in order
69	Idempotency	Convert the write step to an upsert; test that reruns don't corrupt data	Test: rerun-safety test	`fix: make pipeline tasks idempotent`	1	Rerunning the DAG produces the same result
70	Failure handling + review	Add logging/alerting on DAG task failure	—	`feat: add DAG failure logging`	1	Failures are visible, not silent

Weekly review: Reviewer question: does your DAG assume it only ever runs once a day, or would it survive being manually re-triggered mid-afternoon? That's the real idempotency test.

WEEK 11 — Monitoring, Logging, Metrics + Cloud Deployment

Why this matters: A system you can't observe is a system you're flying blind on. This week also moves the project from "works on my laptop" to "works on a real cloud instance," which matters for any resume claim of production-grade work.

Concepts to master (essential): structured logging, logs vs. metrics vs. traces, the four golden signals (latency, traffic, errors, saturation), Prometheus instrumentation, Grafana dashboards, Cloud Run basics.

Resources:

- Prometheus "Querying basics" (https://prometheus.io/docs/prometheus/latest/querying/basics/) plus the `prometheus_client` Python library README for instrumentation.
- Grafana's "Getting Started" — just enough to build one dashboard (https://grafana.com/docs/grafana/latest/getting-started/).
- Google Cloud Run "Quickstart: Deploy a Python service" (https://cloud.google.com/run/docs/quickstarts/deploy-container) — the closest cloud-native match to what you've already built with Docker.

Common mistakes: logging everything at the same verbosity with no structure, so debugging a real incident means grepping unstructured text under pressure.

Connects to Google-scale systems: the four golden signals are literally Google's own SRE terminology — not an analogy, the actual internal vocabulary.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
71	Metrics endpoint	Add `prometheus_client`, expose `/metrics`	—	`feat: add Prometheus metrics endpoint`	2	`/metrics` returns Prometheus-format data
72	Instrumentation	Add request count + latency histogram instrumentation to every route	—	`feat: instrument request latency and count`	2	Real per-endpoint metrics populate
73	Prometheus service	Add Prometheus to compose with a scrape config targeting your app	—	`feat: add Prometheus service and scrape config`	1	Prometheus UI shows scraped metrics
74	Grafana	Add Grafana to compose, connect Prometheus as a data source	—	`feat: add Grafana service`	1	Grafana can query Prometheus
75	Dashboard	Build one dashboard panel showing p50/p99 latency	—	`feat: add latency dashboard panel`	1	Live latency percentiles visible
76	Structured logging	Convert all logging to structured JSON across services; remove stray `print()` calls	Refactor: consistent logging format	`refactor: convert to structured JSON logging`	3	Logs are machine-parseable
77	Cloud deploy + review	Deploy the containerized API to Cloud Run; document the steps	—	`feat: add Cloud Run deployment config`	2	A live, curlable Cloud Run URL exists

Weekly review: You now have a real Grafana panel and a live cloud deployment — both genuinely resume-worthy artifacts. Reviewer question: if p99 latency spiked right now, would your dashboard tell you *why*, or just *that*?

WEEK 12 — System Design, Reliability, Performance

Why this matters: This week doesn't add new technology — it sharpens the two skills actually evaluated in a Google-style interview and in your first six months on a team: designing under constraints, and calmly debugging something on fire.

Concepts to master (essential): back-of-the-envelope capacity estimation, bottleneck identification under load, graceful degradation, defining and enforcing SLOs.

Resources:

- "Designing Data-Intensive Applications," Chapter 1 only (Reliable, Scalable, Maintainable Applications) — the chapter deliberately skipped in Week 9 now lands with real context behind it.
- Google's SRE book, "Embracing Risk" and "Service Level Objectives" chapters, free online (https://sre.google/sre-book/table-of-contents/).

Common mistakes: treating "resilience" as a single feature to bolt on, rather than a property designed into every dependency call (timeouts, fallbacks, circuit breakers) individually.

Connects to Google-scale systems: this week *is* the SRE discipline in miniature — SLOs, graceful degradation, blameless postmortems are daily practice at this scale, not academic abstraction.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
78	Circuit breaker	Add a circuit breaker pattern around the Redis dependency	—	`feat: add circuit breaker for Redis dependency`	1	Redis failures don't cascade into full request failures
79	Fallback path	Add a fallback: if Redis is down, serve from Postgres directly	Test: simulate Redis outage	`feat: add fallback path when cache unavailable`	1	System stays up when the cache is unavailable
80	Load testing	Add a Locust load test; record baseline throughput/latency	—	`test: add Locust load test baseline`	1	Documented baseline numbers
81	Bottleneck fix	Find and fix one real bottleneck surfaced by the load test (e.g., an N+1 query)	—	`perf: fix N+1 query bottleneck`	1	Measurable latency improvement
82	SLOs	Define target latency/error-rate SLOs; add a corresponding Prometheus alert rule stub	—	`feat: define SLOs and alerting rules`	1	Concrete, numeric reliability targets exist
83	Re-test	Re-run the load test after fixes; compare and document results	—	`docs: document performance improvement results`	1	Before/after numbers recorded
84	Cleanup + review	Full tech-debt pass: refactor rough edges accumulated over 11 weeks	Refactor: broad cleanup	`refactor: cleanup tech debt across services`	varies	Codebase reads like a maintained project, not an accretion of features

Weekly review: Reviewer question: pick your worst-designed module from Week 1–3 — would you rebuild it the same way today? If not, that's the actual sign you've learned something.

WEEK 13 — Capstone: Integration, Incident Simulation, Release

Why this matters: Everything so far has been built incrementally. This final stretch integrates it into one coherent, demoable, defensible system — and rehearses the exact skill

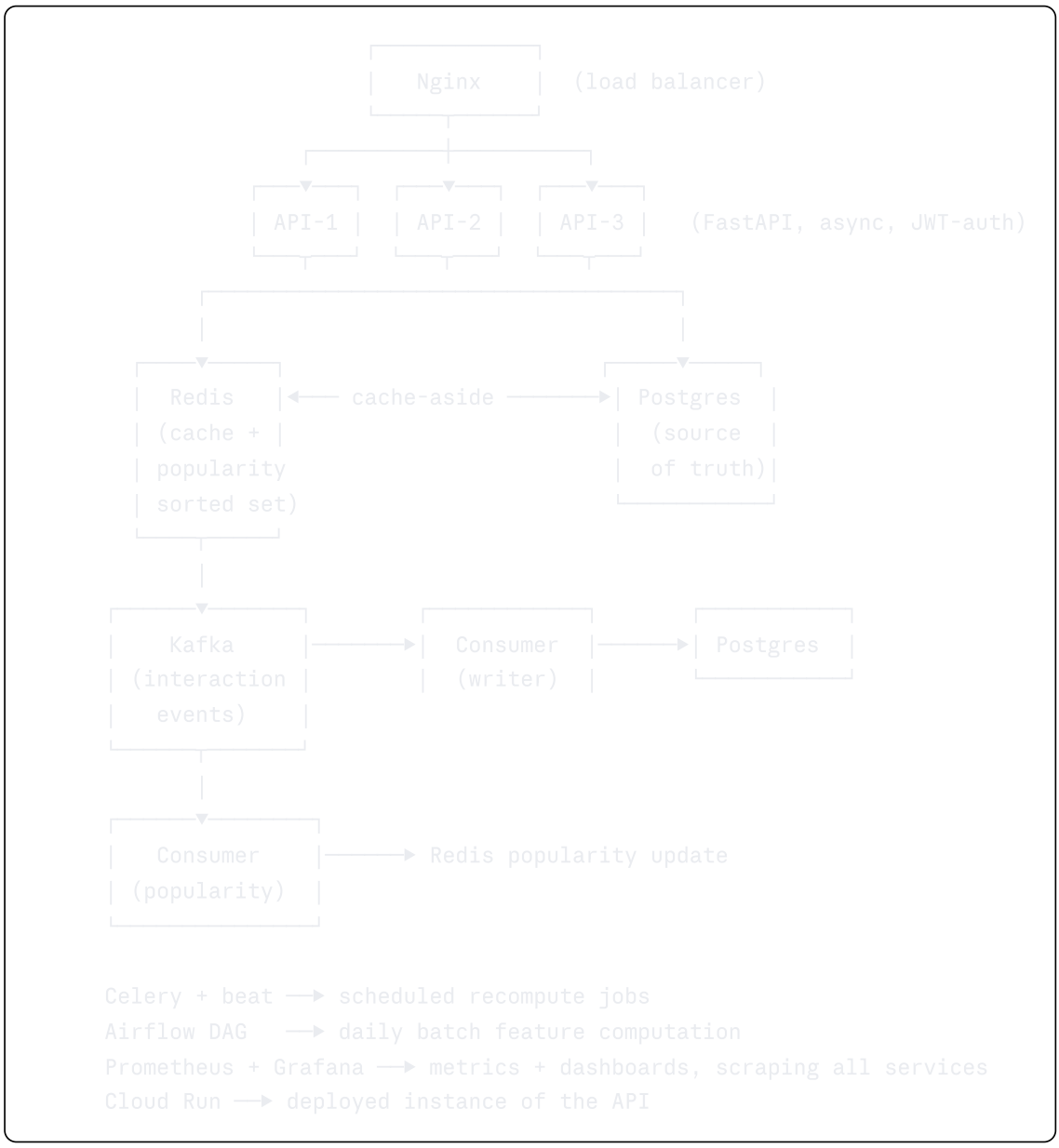
of calmly narrating a production incident, which is what actually gets evaluated in interviews and on the job.

Resources: none new — this week is entirely integration and synthesis of everything already read.

Day	Concept	Coding Task	Test/Refactor	Commit Message	Files	Expected Outcome
85	Incident simulation	Kill Redis mid-load-test; capture the Grafana latency spike; verify graceful degradation holds	Test: full incident simulation	`test: simulate incident and verify degradation`	1	A real, documented failure-and-recovery cycle
86	Postmortem	Write a blameless postmortem of the Day 85 incident, Google SRE style	—	`docs: add incident postmortem`	1	A postmortem doc that reads like a real team wrote it
87	End-to-end tests	Add a full integration test: register → login → interact → score	Test: e2e suite	`test: add end-to-end integration test suite`	2	The entire user journey is test-covered
88	Final docs	Write a full README with architecture diagram and setup instructions	—	`docs: finalize README with architecture overview`	1	Anyone (including a recruiter) can understand the system in 2 minutes
89	Code quality pass	Final lint/type-hint/docstring pass across the codebase	Refactor: quality audit	`chore: final code quality pass`	varies	Codebase would pass a real code review
90	Release + retrospective	Tag `v1.0`; write a short project retrospective (what you'd do differently)	—	`chore: tag v1.0 release and add retrospective`	1	A tagged, presentable, portfolio-ready release

Final Evaluation

Architecture (final state)



Folder structure (final state)

```
cascade/
├── app/
│   ├── main.py
│   ├── routers/
│   │   ├── auth.py
│   │   ├── score.py
│   │   └── interact.py
│   ├── services/
│   │   ├── scoring.py
│   │   └── cache.py
│   ├── repository/
│   │   └── interactions.py
│   ├── schemas/
│   │   └── score.py
│   ├── models/
│   │   └── entities.py
│   └── core/
│       ├── config.py
│       ├── security.py
│       └── logging.py
├── workers/
│   ├── celery_app.py
│   ├── tasks.py
│   └── kafka_consumer.py
├── pipelines/
│   └── dags/
│       └── daily_features_dag.py
├── tests/
│   ├── test_auth.py
│   ├── test_score.py
│   ├── test_cache.py
│   └── test_e2e.py
├── monitoring/
│   ├── prometheus.yml
│   └── grafana/
├── docs/
│   └── adr/
│       ├── 001-cap-tradeoffs.md
│       ├── 002-replication-strategy.md
│       └── 003-sharding-strategy.md
├── docker-compose.yml
├── Dockerfile
└── README.md
```

Technologies used and why

FastAPI (async-native, typed, self-documenting) · PostgreSQL (relational source of truth) · SQLAlchemy 2.0 (typed query layer) · Redis (cache + lightweight ranking structure) · Celery (background task decoupling) · Kafka (durable multi-consumer event log) · Docker/Compose (reproducibility) · Nginx (load balancing) · Airflow (batch orchestration with real dependencies) · Prometheus/Grafana (the four golden signals, made visible) · Cloud Run (the cloud-native deploy target closest to your existing Docker skills).

Skills gained

Layered service architecture, real async I/O reasoning (not cargo-culted), relational schema and index design, stateless auth, container-based reproducibility, cache-aside and invalidation tradeoffs, task-queue idempotency, event-log vs. task-queue reasoning, replica/CAP/sharding intuition backed by a real (if small) implementation, batch pipeline idempotency, observability instrumentation, and — critically — the habit of writing a design doc before code and a postmortem after an incident.

Remaining weaknesses (be honest with yourself here)

Kafka and distributed systems specifically: you'll have working intuition and interview-ready vocabulary, not production battle-scars — those come from time on a real team under real load, not a solo 90-day project. You also won't have touched Kubernetes, service meshes, or multi-region replication — deliberately out of scope; don't let an interviewer's mention of them rattle you, just say plainly what you have and haven't done.

Suggested next projects

1. Add a real two-tower model (you already have this skill) behind the `/score` endpoint, replacing the stub scorer — this is the natural bridge back to your ML work.
2. Add a lightweight A/B testing framework on top of Cascade to compare ranking strategies.
3. Add Kubernetes as a *learning* exercise once this roadmap ends, migrating Compose to a local k3s/minikube cluster.

Suggested interview prep

- System design: "design a recommendation feed like Reels/TikTok" and "design a rate limiter" — both map directly onto what you built.
- Behavioral: have the Day 86 postmortem ready as a real story for "tell me about a time something broke."

Suggested system design problems

Design a URL shortener (classic, tests sharding/caching); design a notification system (tests queues/idempotency); design a news feed ranking service (this is Cascade, described from memory, which is the best possible interview prep).

Suggested LeetCode topics

Given your systems focus, prioritize: hash maps, sliding window, and heap/priority-queue problems (these map to caching/ranking logic you just built) over exotic DP — system design interviews will weigh more heavily than LeetCode for the roles you're targeting anyway.

Master Ranking Table

Topic	Google SWE	ML Infra	AI Engineering	Difficulty	Est. Hours
API Design (FastAPI)	Essential	Essential	Essential	3	8
Software Architecture (layering)	Essential	Essential	Important	4	6
SQL / PostgreSQL	Essential	Essential	Essential	4	12
SQLAlchemy	Important	Essential	Essential	4	8
Authentication	Important	Important	Important	4	6
Docker	Essential	Essential	Essential	3	8
Docker Compose	Important	Essential	Essential	3	5
Async Programming	Essential	Essential	Essential	7	10
Multithreading	Important	Important	Important	6	4
Multiprocessing	Important	Essential	Essential	6	5
Concurrency (theory: GIL etc.)	Essential	Essential	Essential	7	6
Redis / Caching	Important	Essential	Essential	5	8
Message Queues (Celery)	Important	Important	Essential	5	8
Kafka	Important	Essential	Essential	8	12
Event-Driven Architecture	Important	Essential	Essential	6	6
CAP Theorem	Essential	Essential	Important	6	4
Replication	Essential	Important	Important	6	4
Sharding	Essential	Important	Important	7	4
Load Balancing	Essential	Important	Important	5	3
Distributed Systems (general)	Essential	Essential	Important	8	10
Data Pipelines / Scheduling	Important	Essential	Essential	6	8
Monitoring / Logging / Metrics	Essential	Essential	Essential	4	8
Prometheus / Grafana	Important	Important	Important	5	6
GCP Fundamentals	Important	Important	Important	4	6
System Design (integration)	Essential	Essential	Important	8	12
Reliability Engineering	Essential	Essential	Important	7	8

Topic	Google SWE	ML Infra	AI Engineering	Difficulty	Est. Hours
Performance Optimization	Important	Essential	Essential	6	6
Debugging Production Systems	Essential	Essential	Essential	6	6

If you only had time for eight of these: API Design, SQL, Docker, Async Programming, Concurrency theory, CAP Theorem, Monitoring, System Design — that subset alone outperforms the full list studied shallowly, in a Google-style interview.